
Analisa

1. Vanishing Gradient pada Lapisan Awal dan Batch Normalization

Fenomena Vanishing Gradient:

- Vanishing gradient adalah masalah yang terjadi ketika nilai gradien yang dihitung selama backpropagation menjadi sangat kecil (mendekati nol). Hal ini sering terjadi pada jaringan yang lebih dalam dan bisa menyebabkan pelatihan tidak efektif, terutama di lapisan awal.
- Pada jaringan yang dalam, gradien yang dihitung melalui rantai produk menjadi lebih kecil seiring perjalanan kembali ke lapisan awal, menyebabkan lapisan awal tidak mendapatkan informasi yang cukup untuk memperbarui bobot secara efisien.
- Dalam kasus CNN, lapisan konvolusi awal mungkin mengalami vanishing gradient jika fungsi aktivasi seperti sigmoid atau tanh digunakan. Fungsi aktivasi ini memiliki rentang output yang terbatas, sehingga gradiennya bisa sangat kecil.

Cara Memitigasi Vanishing Gradient:

- Gunakan fungsi aktivasi ReLU (Rectified Linear Unit) yang lebih efektif dalam mencegah vanishing gradient dibandingkan fungsi sigmoid atau tanh, karena ReLU memiliki gradien yang tetap (1) ketika nilai input positif.
- Inisialisasi bobot yang baik, misalnya dengan Xavier atau He Initialization, yang membantu gradien agar tidak terlalu kecil atau terlalu besar.
- Batch Normalization (BN): Dapat membantu mengatasi masalah ini dengan menormalisasi output dari setiap lapisan, menjaga nilai distribusi aktivasi dalam batas yang stabil selama pelatihan.

Mengapa Batch Normalization Setelah Lapisan Konvolusi ke-Y Bisa Memperburuk Generalisasi:

- Batch Normalization (BN) sering digunakan untuk menstabilkan pelatihan dengan mengurangi perubahan distribusi aktivasi antar mini-batch. Namun, jika BN diterapkan pada lapisan yang sangat dalam (misalnya lapisan ke-Y), bisa menyebabkan "over-normalization", yang mengurangi keberagaman representasi yang dihasilkan oleh lapisan tersebut.
- Hal ini bisa menghambat model untuk belajar fitur yang lebih kompleks pada lapisan-lapisan lebih dalam dan menyebabkan penurunan generalisasi. Selain itu, BN bisa menambah kompleksitas model dengan menambah parameter yang harus dipelajari, yang kadang-kadang bisa memperburuk kinerja pada data validasi.

Strategi Alternatif:

- Layer Normalization: Normalisasi pada setiap layer individu daripada batch bisa menjadi alternatif untuk BN.
- Residual Networks (ResNet): Menggunakan skip connections (shortcut) yang memungkinkan gradien mengalir lebih bebas tanpa terhambat di lapisan yang lebih dalam.

2. Stagnasi Loss Training dan Penyebab Terkait Learning Rate, Inisialisasi Berat, dan Kompleksitas Model

Tiga Penyebab Potensial:

1. Learning Rate Terlalu Tinggi: Jika learning rate terlalu tinggi, model mungkin melompat-lompat di sekitar solusi optimal dan gagal konvergen, yang menyebabkan loss stagnan di nilai tinggi.

2. Inisialisasi Bobot yang Tidak Tepat: Jika bobot diinisialisasi dengan cara yang buruk, seperti nilai terlalu kecil atau besar, ini bisa menyebabkan kesulitan dalam pembelajaran dan stagnasi loss.
3. Kompleksitas Model yang Terlalu Tinggi atau Terlalu Rendah: Model yang terlalu kompleks dengan terlalu banyak parameter bisa menyebabkan overfitting atau kesulitan dalam pelatihan jika data tidak cukup. Sebaliknya, model yang terlalu sederhana mungkin tidak dapat menangkap pola yang cukup rumit dalam data.

Mengapa Cyclic Learning Rate Membantu Model Keluar dari Local Minima:

- Cyclic Learning Rate (CLR) memungkinkan learning rate berubah secara dinamis selama pelatihan, naik dan turun dalam siklus. Ini membantu model untuk keluar dari local minima (atau saddle points) dan menemukan solusi yang lebih baik daripada jika learning rate tetap statis.
- Dengan mengubah learning rate secara siklik, model bisa bergerak lebih cepat dan lebih efisien ke area solusi yang lebih baik.

Momentum pada Optimizer SGD:

- Momentum pada SGD membantu mengurangi osilasi dalam arah gradien yang tidak perlu dan mempercepat konvergensi. Dengan momentum, pembaruan parameter tidak hanya bergantung pada gradien saat ini, tetapi juga pada gradien masa lalu, memungkinkan model untuk bergerak lebih stabil dan cepat menuju minimum global.

3. Dying ReLU dan Gangguan Aliran Gradien

Fenomena Dying ReLU:

- Dying ReLU terjadi ketika neuron dengan fungsi aktivasi ReLU berhenti memperbarui bobotnya karena outputnya selalu 0 (untuk input negatif). Ini mengarah pada "dead neurons", yang tidak memberikan kontribusi apapun pada pembaruan parameter selama pelatihan.
- Hal ini bisa terjadi jika learning rate terlalu tinggi, sehingga bobot terupdate terlalu besar dan menyebabkan nilai input menjadi sangat negatif, atau jika model tidak memiliki cukup diversitas data yang cukup.

Mengapa Dying ReLU Mengganggu Aliran Gradien:

- Ketika neuron "mati", gradien tidak bisa mengalir kembali ke lapisan sebelumnya. Karena ReLU memiliki gradien 0 untuk input negatif, ini menyebabkan gradien yang melewati neuron tersebut menjadi 0, yang menghambat pembaruan bobot yang diperlukan untuk belajar.

Solusi untuk Dying ReLU:

- Leaky ReLU: Menggunakan fungsi aktivasi Leaky ReLU, yang memperbolehkan gradien mengalir meskipun inputnya negatif (dengan sedikit nilai negatif, biasanya 0.01).
- Parametric ReLU (PReLU): Variasi dari Leaky ReLU yang membiarkan model belajar nilai kemiringan untuk input negatif.

4. Class-Weighted Loss Function Gagal Meningkatkan Kinerja Kelas Spesies X

Mengapa Class-Weighted Loss Tidak Berhasil:

- Class-weighted loss function dirancang untuk memberikan penalti lebih besar pada kelas yang kurang terwakili. Jika ini tidak meningkatkan kinerja Spesies X, ada beberapa penyebab potensial:

- Data yang Tidak Cukup Variasi: Mungkin data untuk kelas Spesies X tidak cukup beragam atau terlalu sedikit untuk membantu model belajar fitur yang relevan.
- Model Overfitting pada Kelas Lain: Jika model terlalu fokus pada kelas lain yang lebih dominan, penambahan bobot tidak akan banyak membantu.
- Desain Model yang Tidak Memadai: Mungkin model tidak cukup dalam untuk menangkap fitur yang diperlukan untuk Spesies X.

Faktor Penyebab Potensial:

1. Ketidakseimbangan Data: Jika satu kelas terlalu dominan, model cenderung mengabaikan kelas minoritas meskipun menggunakan class-weighted loss.
2. Kurangnya Augmentasi pada Kelas Minoritas: Augmentasi yang tidak cukup pada data untuk Spesies X bisa menyebabkan model tidak mempelajari representasi yang cukup baik.
3. Model yang Tidak Memadai: Arsitektur model yang terlalu sederhana atau terlalu rumit dapat mengarah pada kesulitan dalam mengklasifikasikan kelas minoritas.

5. Overfitting pada Model CNN yang Terlalu Kompleks

Fenomena Overfitting:

- Overfitting terjadi ketika model terlalu sesuai dengan data pelatihan sehingga kehilangan kemampuan untuk menggeneralisasi ke data yang belum terlihat. Akibatnya, model dapat memiliki akurasi pelatihan yang tinggi, tetapi akurasi validasi yang rendah.
- Peningkatan kompleksitas model sering kali tidak meningkatkan generalisasi, bahkan bisa memperburuknya, terutama jika dataset terlalu kecil atau tidak cukup beragam.

Kenapa Penambahan Kapasitas Model Tidak Selalu Meningkatkan Generalisasi:

- Model yang lebih kompleks mungkin memiliki lebih banyak parameter dan lebih banyak kapasitas untuk menghafal daripada untuk belajar pola yang benar.
- Jika data pelatihan tidak cukup atau tidak bervariasi, model yang lebih kompleks cenderung memasukkan noise yang tidak perlu, mengarah ke overfitting.

Tiga Kesalahan Desain Arsitektur:

1. Jumlah Lapisan yang Terlalu Banyak: Model dengan lapisan konvolusi yang terlalu banyak bisa menyebabkan overfitting, terutama jika dataset kecil.
2. Ukuran Batch Terlalu Besar: Ukuran batch yang besar dapat mengurangi kemampuan model untuk belajar dengan tepat dan menyebabkan gradien yang tidak tepat.
3. Kurangnya Regularisasi: Tidak menggunakan teknik regularisasi seperti Dropout, L2 Regularization, atau Batch Normalization bisa menyebabkan model overfit.

Rangkuman:

- Vanishing Gradient: Gunakan ReLU dan inisialisasi bobot yang baik untuk mencegah masalah ini.
- Learning Rate: Perbaiki dengan Cyclic Learning Rate dan momentum untuk membantu menghindari local minima.
- Dying ReLU: Gunakan Leaky ReLU atau PReLU untuk mencegah masalah ini.
- Class-Weighted Loss: Gunakan augmentation untuk kelas minoritas dan pastikan model cukup dalam untuk mengenali pola kelas tersebut.
- Overfitting: Gunakan regularisasi dan sederhanakan model agar tidak berlebihan.